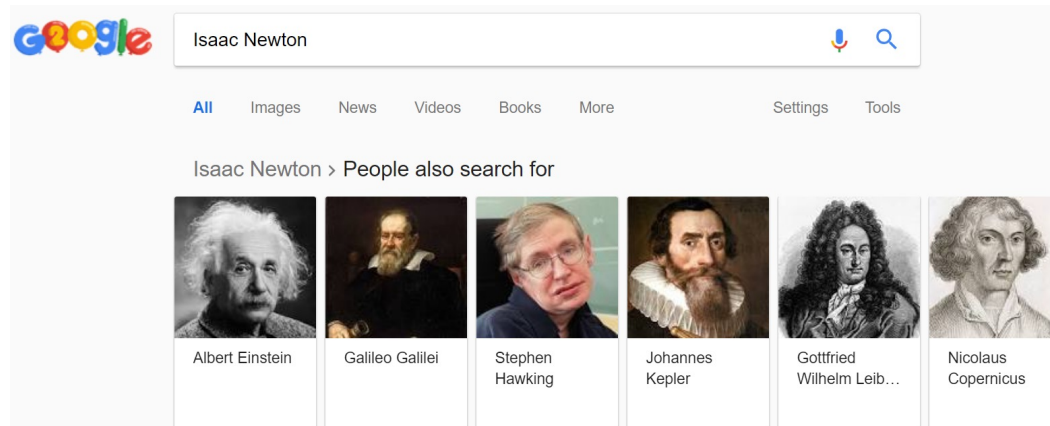
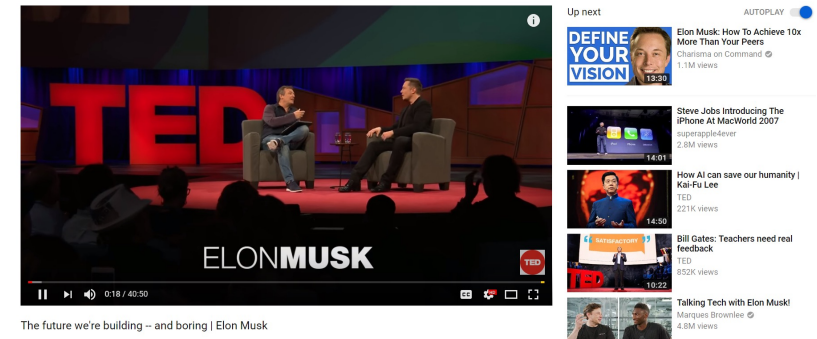


Cloud Based Data Engineering

Lecture 9: Recommendation Systems

Recommender Systems

A **recommender system** (RS) helps users that have no sufficient competence or time to evaluate the, potentially overwhelming, number of alternatives offered by a web site. In their simplest form, RSs recommend to their users personalized and **ranked lists of items**

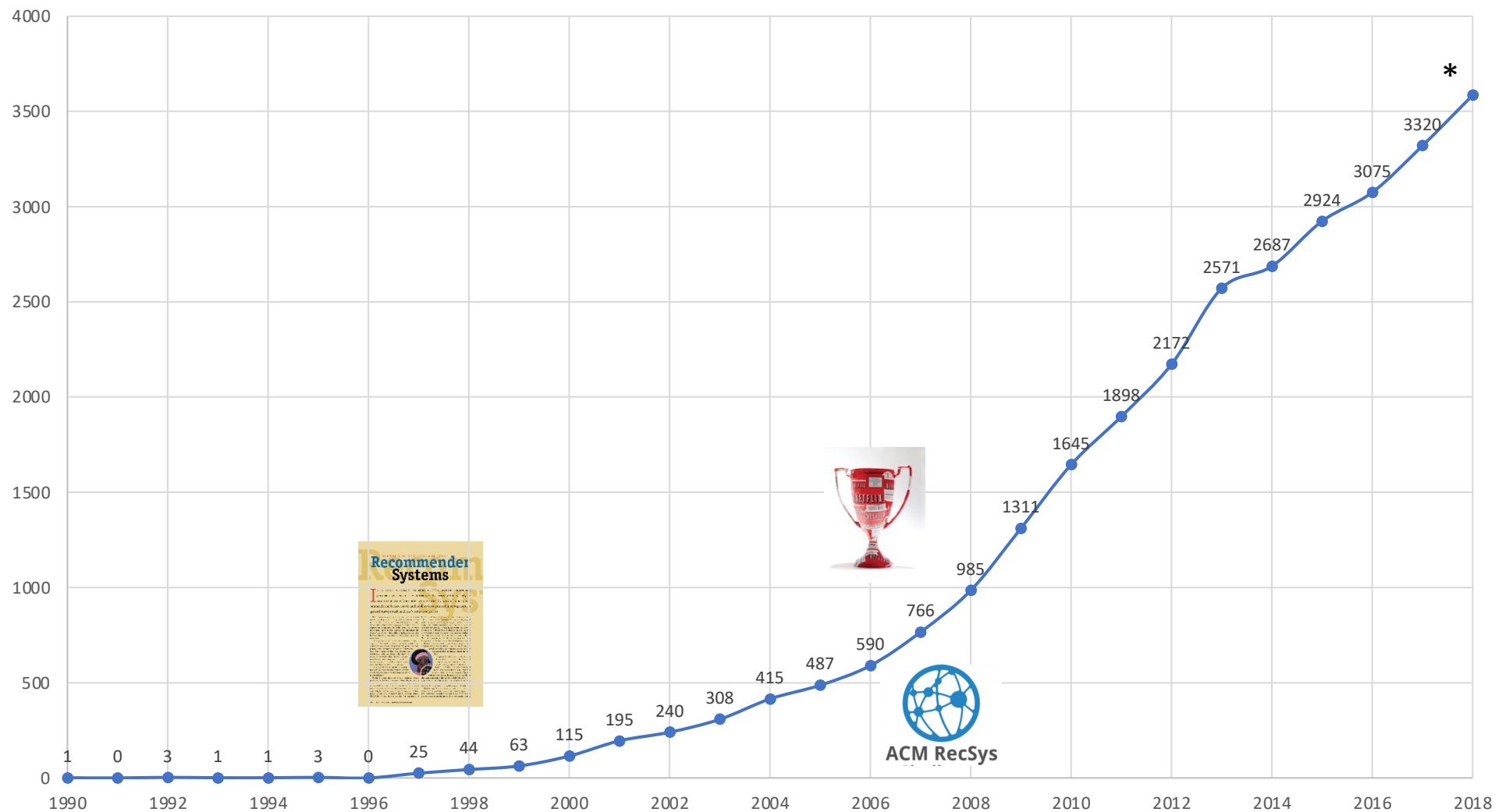


The Impact of Recommendation Systems

- 35% of the purchases on **Amazon** are the result of their recommender system.
- Recommendations are responsible for 70% of the time people spend watching videos on **YouTube**.
- 75% of what people are watching on Netflix comes from recommendations, according to **McKinsey**

The Rise of the Recommender System

Papers in Microsoft Academic



Recommendation Models



Model	Commonness	Used By:															
		Jinni	Taste Kid	Nanocrowd	Clerkdogs	Criticker	IMDb	Flixster	Movielens	Netflix	Shazam	Pandora	LastFM	YooChoose	Think Analytics	Itunes	Amazon
Collaborative Filtering	☺	✓				✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Content-Based Techniques	☺	✓	✓	✓	✓		✓		✓			✓	✓	✓	✓		✓
Knowledge-Based Techniques	☺	✓	✓	✓	✓		✓				✓				✓		
Stereotype-Based Recommender Systems	☺	✓	✓	✓	✓		✓							✓	✓		
Ontologies and Semantic Web Technologies for Recommender Systems	☺	✓		✓								✓					
Community Based Recommender Systems	☺	✓		✓		✓			✓	✓				✓	✓		
Demographic Based Recommender Systems										✓							
Context Aware Recommender Systems	☺	✓		✓	✓	✓								✓	✓		
Conversational/Critiquing Recommender Systems	☺	✓													✓		
Hybrid Techniques	☺	✓						✓		✓	✓	✓					

Challenges

Cold start / Data sparsity

- Collaborative filtering methods recommend items based on users' past preferences
- New users will need to rate several items to enable the system to capture their preferences accurately
- For new systems, the initial state is an empty dataset

Challenges

“Gray sheep”

Refers to the users whose opinions do not consistently agree or disagree with any group of people, and thus do not benefit from collaborative filtering.

Scalability

As the numbers of users and items grow, traditional CF algorithms will suffer serious scalability problems.

The Netflix Problem

- The Netflix problem was introduced in 2006
- Hundreds of thousands of users across thousands of movies
- Predict user rating (1-5)
- Many missing data = **sparse** matrix (a lot of missing values or zeros)

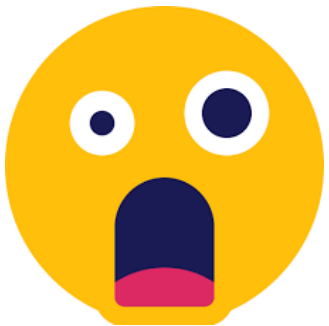
	7					6
	7	6	3		4	
	4	3				
4	2					
				3	2	4

0	7	0	0	0	0	6
0	7	6	3	0	4	0
0	4	3	0	0	0	0
4	2	0	0	0	0	0
0	0	0	0	3	2	4

The Netflix Problem - Rules

Terms and Conditions in a Nutshell

- Contest begins October 2, 2006 and continues through at least October 2, 2011.
- Contest is open to anyone, anywhere (except certain countries listed below).
- You have to register to enter.
- Once you register and agree to these Rules, you'll have access to the Contest training data and qualifying test sets.
- To qualify for the \$1,000,000 Grand Prize, the accuracy of your submitted predictions on the qualifying set must be at least 10% better than the accuracy Cinematch can achieve on the same training data set at the start of the Contest.
- To qualify for a year's \$50,000 Progress Prize the accuracy of any of your submitted predictions that year must be less than or equal to the accuracy value established by the judges the preceding year.
- To win and take home either prize, your qualifying submissions must have the largest accuracy improvement verified by the Contest judges, you must share your method with (and non-exclusively license it to) Netflix, and you must describe to the world how you did it and why it works.



https://en.wikipedia.org/wiki/Netflix_Prize

Netflix Prize

COMPLETED

[Home](#)[Rules](#)[Leaderboard](#)[Update](#)

Leaderboard

Showing Test Score. [Click here to show quiz score](#)

Display top leaders.

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
------	-----------	-----------------	---------------	------------------

[Grand Prize](#) - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos

1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11

20 min
later

The Prize Goes To ...

- Once a team succeeded to improve the RMSE by 10%, the jury issue a last call, giving all teams 30 days to send their submissions.
- On July 25, 2009 the team "The Ensemble" achieved a 10.09% improvement.
- After some dispute ...



Approaches

Recommender systems can be loosely broken down into three categories:

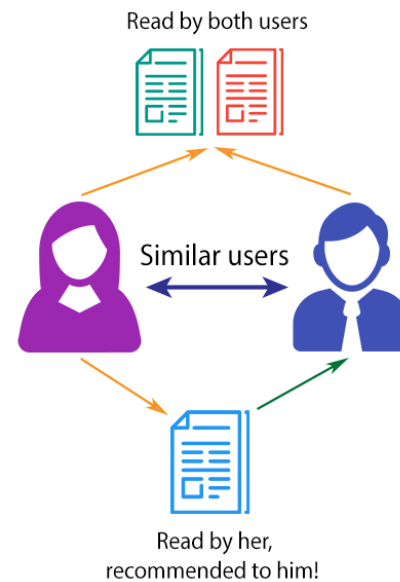
content-based systems

collaborative filtering systems

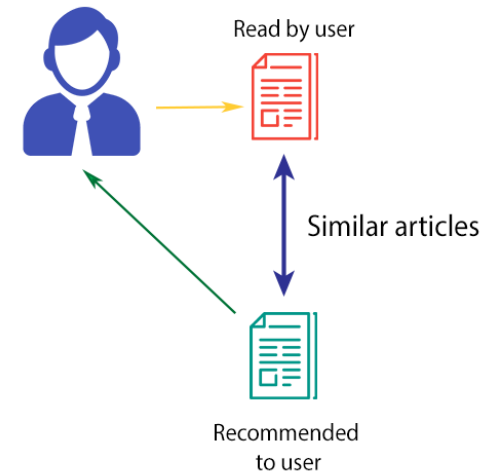
hybrid systems

(a combination of the other two).

COLLABORATIVE FILTERING



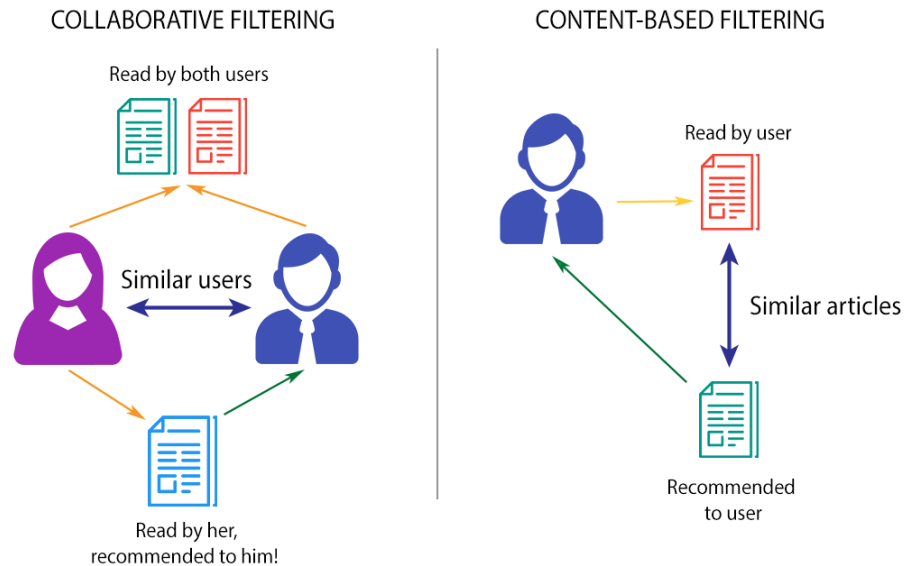
CONTENT-BASED FILTERING



Approaches

Collaborative filtering approach builds a model from a user's past behaviors (items previously purchased or selected and/or numerical ratings given to those items) as well as similar decisions made by other users.

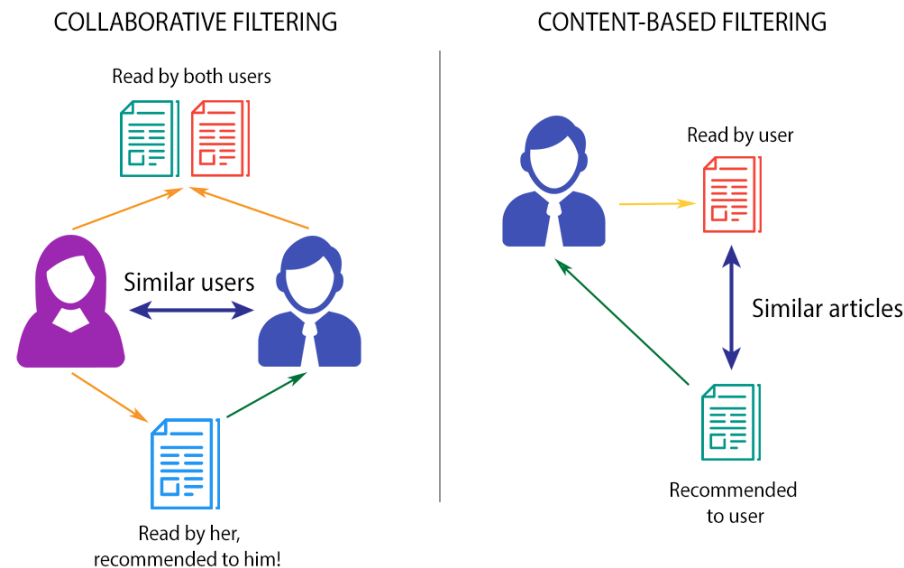
This model is then used to predict items (or ratings for items) that the user may have an interest in.



Approaches

Content based approach utilizes a series of discrete characteristics of an item in order to recommend additional items with similar properties.

Hybrid approach combines the previous two approaches. Most businesses probably use hybrid approach in their production recommender systems.



Collaborative Filtering

Collaborative Filtering is a mathematical method to find the predictions about how users can rate a particular item based on ratings of other similar users.

Typical Collaborative Filtering involves 4 different stages:

Data Collection — Collecting user behaviors and associated data items

Data Processing — Processing the collected data

Recommendation Calculation — Calculate referrals based on processed data

Result Derivation — Extract the similarity and return the top N results

Explicit and Implicit Feedbacks

Explicit Feedback is ratings given by users, which Netflix is collecting to make recommendations to their customers after they provide ratings to the movies they have watched.

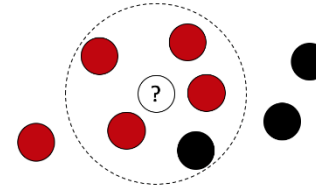


Implicit Feedback is (less straightforward) based on the users' interactions with the platform, ranging from clicks, views, likes and purchases. Spotify makes use of Implicit Feedbacks to implement its recommendation system.

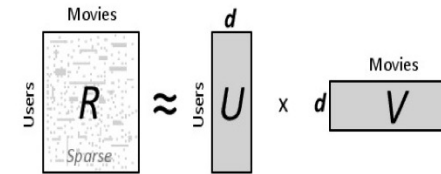


Collaborative Filtering Techniques

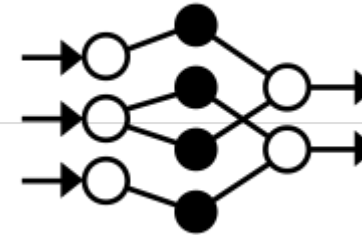
Nearest Neighbor (Pandas)



Matrix Factorization (Spark)



Deep Learning NN (Keras) – not here

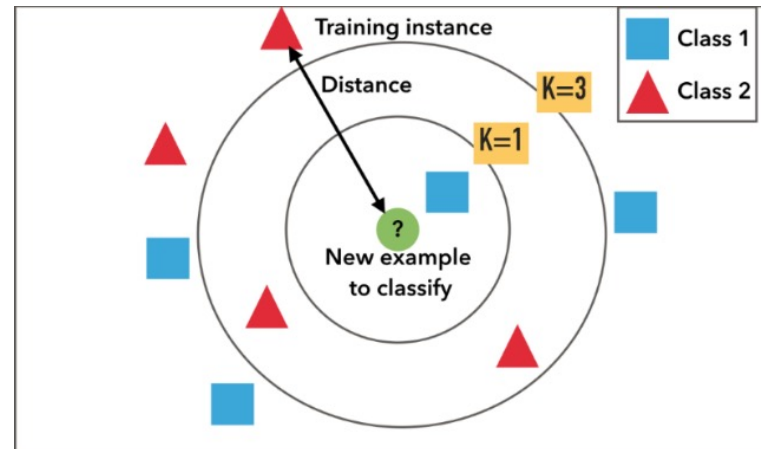


Technique 1 – K Nearest Neighbors

KNN item-Based Collaborative Filtering

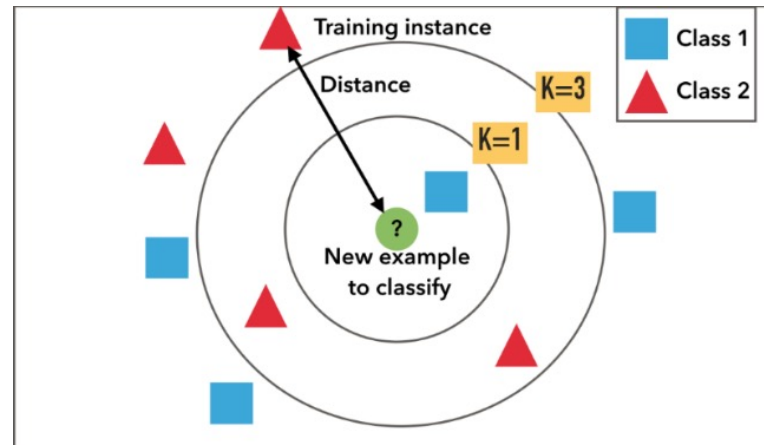
KNN is a **non-parametric**, **lazy** learning method.

It uses a database in which the data points are separated into several clusters to make inference for new samples.

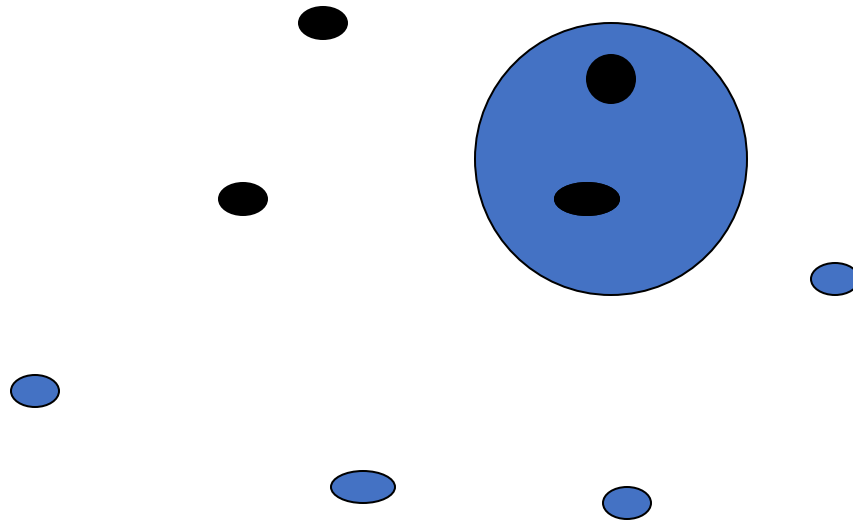


KNN item-Based Collaborative Filtering

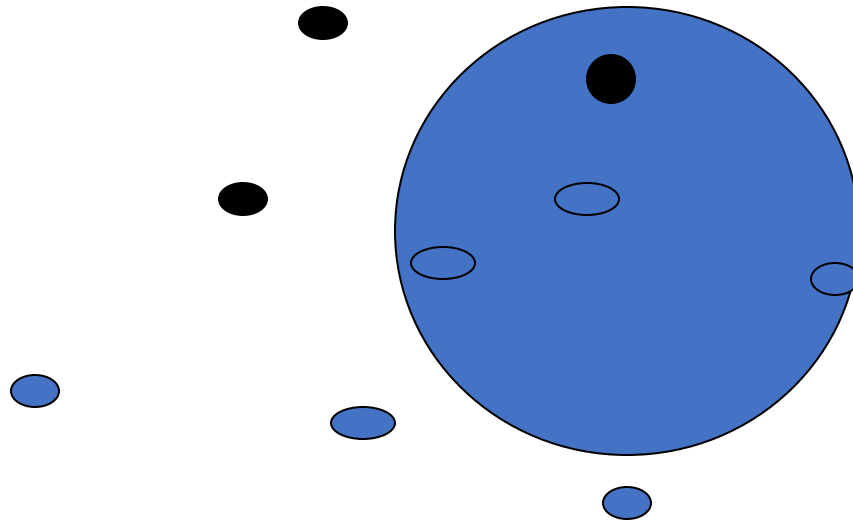
When KNN makes inference about a datapoint, KNN will calculate the “distance” between the target and every other points in its database, then it ranks its distances and returns the top K nearest neighbor points as the most similar recommendations.



1-Nearest Neighbor



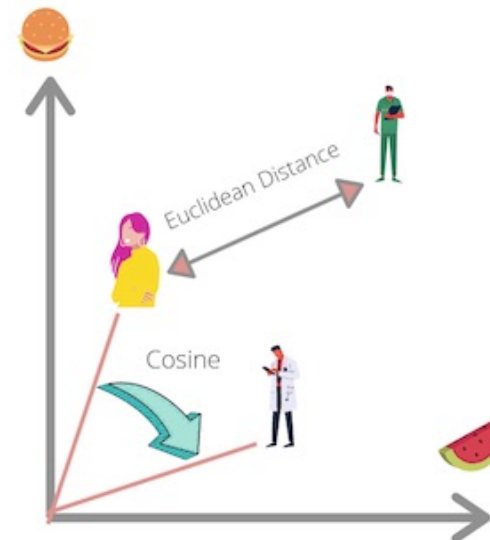
3-Nearest Neighbor



KNN item-Based Collaborative Filtering

How to find “close” items?

1. **Euclidean distance** - unhelpful in high dimensions because all vectors are almost equidistant to the search query vector
2. **Cosine similarity** - a measure of similarity between two sequences of numbers
3. **Locality sensitive hashing** – not here



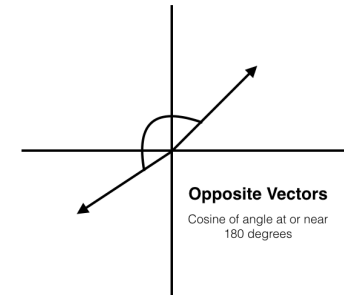
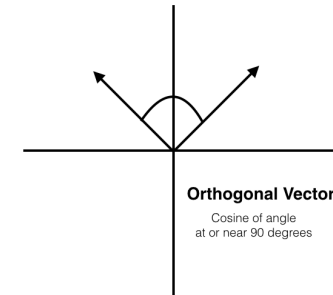
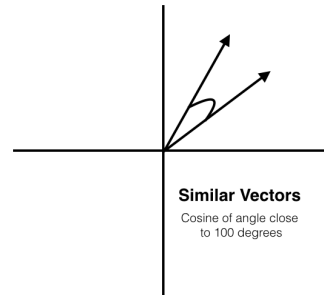
Cosine Similarity

$$\text{Cosine}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

$$A = (a_1, a_2, \dots, a_n), \quad B = (b_1, b_2, \dots, b_n)$$

$$\|A\| = \sqrt{a_1^2 + \dots + a_n^2}$$

$$\|B\| = \sqrt{b_1^2 + \dots + b_n^2}$$



Cosine Similarity

$$\text{Cosine}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

$$A = (3, 2, 0, 5)$$

$$B = (1, 0, 0, 0)$$

$$A \cdot B = 3 \cdot 1 + 2 \cdot 0 + 0 \cdot 0 + 5 \cdot 0 = 3$$

$$\|A\| = \sqrt{3^2 + 2^2 + 0^2 + 5^2} = 6.164$$

$$\|B\| = \sqrt{1^2 + 0^2 + 0^2 + 0^2} = 1$$

$$\text{Cosine}(A, B) = \frac{3}{6.164 \cdot 1} = 0.486$$

Issue with the user-based k NN CF

- The problem with the user-based formulation of collaborative filtering is the lack of scalability:
 - it requires the real-time comparison of the target user to all user records in order to generate predictions.
- A variation of this approach that remedies this problem is called **item-based CF**.

KNN item-Based Collaborative Filtering

MovieLens dataset contains 27,753,444 ratings and 1,108,997 tag applications across 58,098 movies. These data were created by 283,228 user and the ratings are on a scale from 1 to 5.

Movies

movieId	title
1	Toy Story (1995)
2	Jumanji (1995)
3	Grumpier Old Men (1995)
4	Waiting to Exhale (1995)
5	Father of the Bride Part II (1995)

Ratings

userId	movieId	rating
1	307	3.5
1	481	3.5
1	1091	1.5
1	1257	4.5
1	1449	4.5

Introduction to Technique 2 – Matrix Factorization

Introduction

A common problem faced by internet companies is that of recommending new products to users in personalized settings

- Amazon's product recommender system
- Netflix movie recommendations
- Streaming Services

The key factor of recommending with big-data is **Matrix Factorization**.

Matrix Factorization

A family of mathematical operations for matrices in linear algebra.

- A factorization of a matrix into a product of matrices.
- Matrix factorization algorithms work by **decomposing** the user-item interaction matrix into the product of two **lower dimensionality rectangular matrices**.

		Item			
		W	X	Y	Z
User	A		4.5	2.0	
	B	4.0		3.5	
	C		5.0		2.0
	D		3.5	4.0	1.0

Rating Matrix

=

A	1.2	0.8
B	1.4	0.9
C	1.5	1.0
D	1.2	0.8

User Matrix

X

	W	X	Y	Z
	1.5	1.2	1.0	0.8
	1.7	0.6	1.1	0.4

Item Matrix

Matrix Factorization

Given matrix R of ratings, factorize it into two matrices U and P such that $U^T P \approx R$.

(some versions are $UP^T \approx R$)

	Item			
	W	X	Y	Z
User A		4.5	2.0	
User B	4.0		3.5	
User C		5.0		2.0
User D		3.5	4.0	1.0

Rating Matrix

=

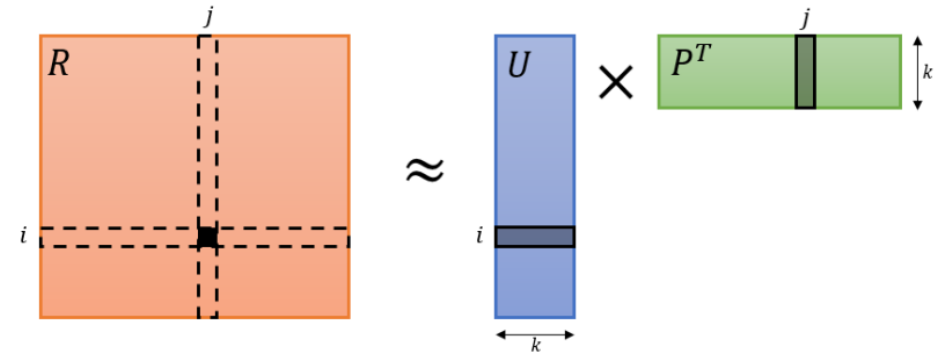
	W	X
User A	1.2	0.8
User B	1.4	0.9
User C	1.5	1.0
User D	1.2	0.8

User Matrix

X

	Item			
	W	X	Y	Z
	1.5	1.2	1.0	0.8
	1.7	0.6	1.1	0.4

Item Matrix



Cost Functions

Mean Squared Error (MSE)

$$\frac{1}{n} \cdot \sum_{i=1}^n (Y_i - \tilde{Y}_i)^2$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{n} \cdot \sum_{i=1}^n (Y_i - \tilde{Y}_i)^2} = \sqrt{MSE}$$

Matrix Factorization

The goal is to find two matrices U, P such that it minimizes the cost function:

$$Cost = (R - U \cdot P^T)^2 + \lambda(U^2 + P^2)$$

MSE Function

Regularization

MSE function - distance measure between the original rating matrix R and its **approximation**
 $U \cdot P^T$

$$MSE = \frac{\sum_{i=1}^n (True - Predicted)^2}{n}, \quad RMSE = \sqrt{MSE}$$

Regularization term - added to govern a generalized solution (to prevent overfitting to some local noisy effects on ratings).

Matrix Factorization

We want to minimize the cost function and the problem become an **optimization** problem.

This optimization problem is naturally solved by machine learning techniques:

1. Gradient Descent (not here)
2. Singular Value Decomposition (SVD)
3. Alternating Least Squares (ALS) - Approximation

Singular Value Decomposition - SVD

SVD is a matrix decomposition technique that has mathematically originated from linear algebra:

$$R = U \cdot P \cdot V^T$$

- P is a diagonal matrix having the singular values of A on its diagonal.
- U and V are orthonormal matrices (their columns are orthogonal, and their norm equals 1)
- SVD solution is unique

We can refer R as the rating matrix and use this method for decomposition.

Singular Value Decomposition - SVD

What's wrong?

1. High Computational Complexity
2. The original matrix must be fully dense (no missing values allowed)
3. This is not the case in most streaming and cloud services



Technique 2 – Matrix Factorization and ALS

Alternating Least Squares (ALS)

The goal of Alternating Least Squares is to find two matrices, U and P , such that their product is approximately equal to the original matrix of users and products.

If we fix P and optimize for U alone, the problem is simply reduced to the problem of linear regression.

In this case, we want to optimize both matrices.

Since it is an approximation algorithm, the solution is not unique

Alternating Least Squares (ALS)

The goal of Alternating Least Squares is to find two matrices, U and P , such that their product is approximately equal to the original matrix of users and products.

Once such matrices have been found, we are able to predict what user i will think of product j by multiplying row i of U with row j of P .

	Product 1	Product 2	Product 3
User 1	0.5	?	4
User 2	1	3	5

Alternating Least Squares (ALS)

Consider the following matrix which holds opinions of two users on three products:

	Product 1	Product 2	Product 3
User 1	0.5	?	4
User 2	1	3	5

We wish to be able to **predict how user 1 will rate product two.**

This is where Alternating Least Squares (ALS) comes in.

Computing U and P in ALS

The method is called “alternating” since the following two steps are repeated:

1. Fix matrix U and find the optimal matrix P
2. Fix matrix P and find the optimal matrix U

	Product 1	Product 2	Product 3
User 1	0.5	?	4
User 2	1	3	5

$$= \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \cdot [p_1 \quad p_2 \quad p_3]$$

Computing U and P in ALS

We start by fixing matrix U. I could have picked any values for U, but I chose 1's to make the maths easier.

	Product 1	Product 2	Product 3
User 1	0.5	?	4
User 2	1	3	5

$$= \begin{bmatrix} 1 \\ 1 \end{bmatrix} \cdot [p_1 \quad p_2 \quad p_3]$$

If we multiply this out, we get 5 equations which involve the components of P:

$$p_1 = 0.5, p_1 = 1, p_2 = 3, p_3 = 4, p_3 = 5$$

There is a lone equation which governs the second component of P, thus we set that equal to 3.

Computing U and P in ALS

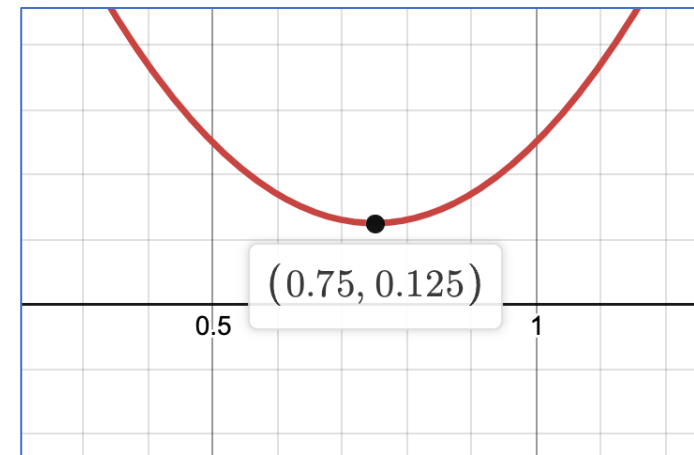
We need to select the remaining two components such that the mean squared error is minimized:

$$p_1^* = (0.5 - p_1)^2 + (1 - p_1)^2 = 2p_1^2 - 3p_1 + \frac{5}{4}$$

What minimize p_1^* ?

$$\frac{\partial p_1^*}{\partial p_1} = 4p_1 - 3 = 0$$

$$p_1 = 0.75$$



Computing U and P in ALS

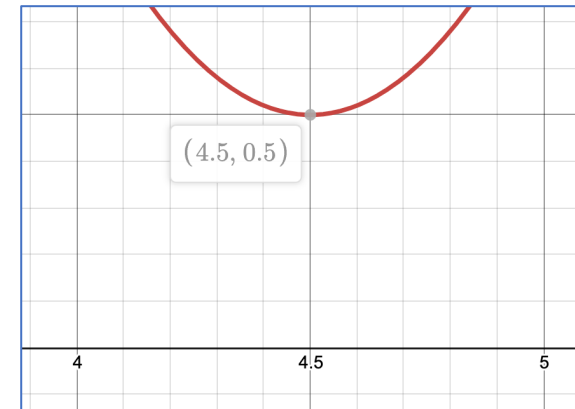
We need to select the remaining two components such that the mean squared error is minimized:

$$p_3^* = (4 - p_3)^2 + (5 - p_3)^2 = 2p_3^2 - 18p_3 + 41$$

What minimize p_3^* ?

$$\frac{\partial p_3^*}{\partial p_3} = 4p_3 - 18 = 0$$

$$p_3 = 4.5$$



Computing U and P in ALS

By solving all the equations, we get the matrix P:

$$P = [0.75, 3, 4.5]$$

Now we keep P fixed and optimize for matrix U. This similarly gives us the following equations for U:

$$0.5 = 0.75u_1$$

$$4 = 4.5u_1$$

$$1 = 0.75u_2$$

$$3 = 3u_2$$

$$5 = 4.5u_2$$

Computing U and P in ALS

Repeating this process, we will, at some point, converge upon the optimal matrices U and P.

Solving those systems, we get:

$$P = [0.75, 2.5431, 4.7999]$$

$$U = \begin{bmatrix} 0.7461 \\ 1.7966 \end{bmatrix}$$

We are now able to predict what user 1 thinks of product 2. By multiplying the first component of U by the second component of P we reach an estimate of:

$$0.7461 \cdot 2.5431 = 1.897$$

How to Build Collaborative Filtering using ALS?

Step 1: Load data

Step 2: Split data into train, validation and test sets.

Train - The sample of data used to fit the model.

Validation - The evaluation becomes more biased as skill on the validation dataset is incorporated into the model configuration. (“semi-testing” your model)

Test - The sample of data used to provide an unbiased evaluation of a final model fit on the training dataset.

How to Build Collaborative Filtering using ALS?

Step 3: ALS Model Selection

To find the optimal hyperparameters, use grid search and select the best model.

Step 4: Learning Curve

As we increased number of iterations in training ALS, we can see how RMSE changes and whether model is overfitted.

Step 5: Make recommendations 😊